

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 5: Revisão do Projeto e Tela de Editar Contato

Do que já construímos ao U de CRUD (Update)

Teoria: CRUD, persistência em memória e Post/Redirect/Get

Objetivos da aula

1

Revisar o projeto construído até aqui

Tela inicial, listagem de contatos, MVC e Bootstrap.

2

Entender o conceito de CRUD

E identificar o que já temos e o que falta no nosso projeto.

3

Resolver onde os dados "moram"

Criar um repositório em memória que sobrevive entre requisições.

4

Construir a tela de Editar Contato

Do clique em "Editar" até salvar a alteração — na prática.

Agenda da aula

- 1 Recapitulando o que já construímos
- 2 Teoria: o que é CRUD
- 3 O problema: onde os dados moram entre requisições
- 4 Solução: um repositório em memória (DAO)
- 5 Tela de Editar Contato: requisitos e mockup
- 6 Teoria: passando o ID de uma tela para outra
- 7 Prática: Controller, View e o padrão Post/Redirect/Get

O que já construímos

Tela Inicial	index.jsp com Bootstrap, imagem/ilustração e botão "Acessar"
Tela Agenda de Contatos	agenda.jsp — grid com ID, Nome, Fone, E-mail e Opções (Editar/Excluir)
Model	Contato.java — id, nome, telefone, email
Controller	AgendaServlet.java — lista os contatos e encaminha para a View
Padrão MVC	Aplicado desde o primeiro commit do projeto oficial

O fluxo MVC que já temos rodando



Hoje vamos adicionar um novo caminho a esse fluxo: o clique em "Editar".

O que é CRUD

CRUD é o acrônimo das quatro operações básicas que praticamente todo sistema com dados precisa oferecer:

C

Create

Criar

Ainda não temos

R

Read

Ler / Listar

✓ Feito (agenda.jsp)

U

Update

Atualizar

 Hoje!

D

Delete

Excluir

Próxima aula

Onde os dados moram entre requisições?

Lembram do HTTP stateless (Aula 1)?

O AgendaServlet cria a lista de contatos DENTRO do doGet — ou seja, uma lista nova a cada requisição. Se editarmos um contato, a alteração se perde no próximo clique, porque a variável não sobrevive entre uma requisição e outra.

```
// dentro do doGet — recriado a cada requisição!  
List<Contato> contatos = new ArrayList<>();  
contatos.add(new Contato(1, "Ana Beatriz Souza", ...));  
contatos.add(new Contato(2, "Carlos Eduardo Lima", ...));
```

Um repositório em memória (DAO)

A pasta "dao" já estava reservada desde a Aula 3 — hoje ela ganha seu primeiro conteúdo, mesmo antes do banco de dados chegar.

Lista estática

Um `List<Contato> static` — pertence à classe, não ao método. Sobrevive enquanto a aplicação estiver no ar.

Ponto único de acesso

Todo Servlet que precisar de contatos passa pelo `ContatoDAO` — ninguém mais cria listas soltas.

Preparando o terreno

Quando o JDBC chegar, só trocamos o "por dentro" do DAO — quem usa (os Servlets) nem percebe.

1 ContatoDAO: nosso repositório em memória

src/main/java/br/edu/faculdade/dao/ContatoDAO.java

```
public class ContatoDAO {
    private static List<Contato> contatos = new ArrayList<>();

    static {
        contatos.add(new Contato(1, "Ana Beatriz Souza", "(35) 99911-2233", "ana.souza@email.com"));
        contatos.add(new Contato(2, "Carlos Eduardo Lima", "(35) 98822-1144", "carlos.lima@email.com"));
    }

    public static List<Contato> listar() {
        return contatos;
    }

    public static Contato buscarPorId(int id) {
        for (Contato c : contatos) {
            if (c.getId() == id) return c;
        }
        return null;
    }

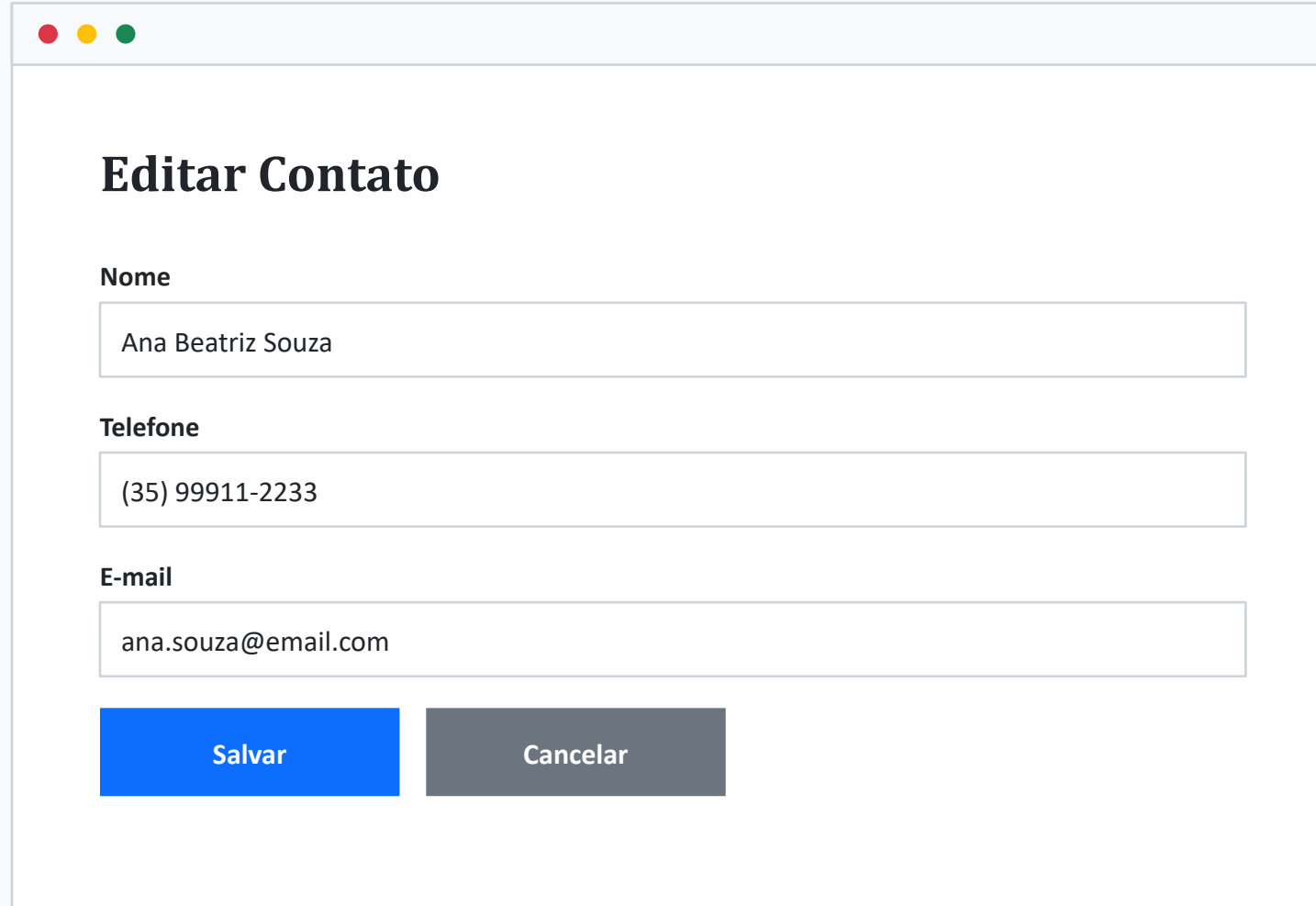
    public static void atualizar(Contato atualizado) {
        for (Contato c : contatos) {
            if (c.getId() == atualizado.getId()) {
                c.setNome(atualizado.getNome());
                c.setTelefone(atualizado.getTelefone());
                c.setEmail(atualizado.getEmail());
            }
        }
    }
}
```

Requisitos da Tela Editar Contato

- 1 Formulário já preenchido com os dados atuais do contato clicado
- 2 Campos: Nome, Telefone e E-mail (editáveis)
- 3 O ID do contato viaja junto, mas não é editável
- 4 Botão "Salvar" grava a alteração e volta para a Agenda
- 5 Botão "Cancelar" volta para a Agenda sem salvar nada

A tela reaproveita o mesmo endpoint "editar-contato": GET mostra o formulário preenchido, POST salva a alteração.

Editar Contato



The image shows a web form titled "Editar Contato" (Edit Contact). The form is contained within a window-like frame with three colored dots (red, yellow, green) in the top-left corner. The form itself has a title "Editar Contato" and three input fields: "Nome" (Name) with the value "Ana Beatriz Souza", "Telefone" (Phone) with the value "(35) 99911-2233", and "E-mail" (Email) with the value "ana.souza@email.com". At the bottom of the form are two buttons: a blue "Salvar" (Save) button and a gray "Cancelar" (Cancel) button.

Editar Contato

Nome

Ana Beatriz Souza

Telefone

(35) 99911-2233

E-mail

ana.souza@email.com

Salvar **Cancelar**

Campos com "form-control" e valor pré-preenchido via `#{contato.nome}`

Como o ID viaja de uma tela para outra

Precisamos dizer ao Servlet QUAL contato editar. A forma mais simples é anexar o ID na própria URL, como query string — lembram do bloco de HTTP (Aula 1)?

agenda.jsp:

```
<a href="editar-contato?id=${c.id}">Editar</a>
```

EditarContatoServlet.java:

```
int id = Integer.parseInt(request.getParameter("id"));
```

Note que é o MESMO request.getParameter() que já usamos para ler campos de formulário na Aula 3 — a query string é só outra forma de chegar até o Servlet.

O FLUXO

O caminho completo da edição



GET mostra o formulário · POST salva e redireciona — dois métodos, um único endpoint.

2

Atualizar o botão Editar na agenda.jsp

```
<a href="editar-contato?id=${c.id}" class="btn btn-warning btn-sm">Editar</a>
```

Antes o botão não tinha href — agora ele monta a URL com o ID de cada contato do laço c:forEach.

3

Controller: buscar o contato (doGet)

src/main/java/br/edu/faculdade/controller/EditarContatoServlet.java

```
@WebServlet("/editar-contato")
public class EditarContatoServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws Exception {
        int id = Integer.parseInt(request.getParameter("id"));
        Contato contato = ContatoDAO.buscarPorId(id);

        request.setAttribute("contato", contato);
        request.getRequestDispatcher("editar-contato.jsp").forward(request, response);
    }
}
```

Igual ao CadastroAlunoServlet da Aula 3 — lê um parâmetro, monta um objeto, coloca na requisição e faz forward. O padrão se repete.

4

View: editar-contato.jsp

src/main/webapp/editar-contato.jsp

```
<div class="container mt-4">
  <h1>Editar Contato</h1>
  <form action="editar-contato" method="POST">
    <input type="hidden" name="id" value="${contato.id}">

    <div class="mb-3">
      <label class="form-label">Nome</label>
      <input type="text" name="nome" class="form-control"
        value="${contato.nome}" required>
    </div>
    <div class="mb-3">
      <label class="form-label">Telefone</label>
      <input type="text" name="telefone" class="form-control"
        value="${contato.telefone}" required>
    </div>
    <div class="mb-3">
      <label class="form-label">E-mail</label>
      <input type="email" name="email" class="form-control"
        value="${contato.email}" required>
    </div>

    <button type="submit" class="btn btn-primary">Salvar</button>
    <a href="agenda" class="btn btn-secondary">Cancelar</a>
  </form>
</div>
```


5

Controller: salvar a alteração (doPost)

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws Exception {
    int id = Integer.parseInt(request.getParameter("id"));
    Contato atualizado = new Contato(id,
        request.getParameter("nome"),
        request.getParameter("telefone"),
        request.getParameter("email"));

    ContatoDAO.atualizar(atualizado);
    response.sendRedirect("agenda");
}
```

POR QUE `sendRedirect()` E NÃO `forward()`?

`forward()` reaproveita a MESMA requisição POST — se o usuário der F5 na página de resultado, o navegador reenvia o POST e duplica a ação.

`sendRedirect()` manda o navegador fazer uma NOVA requisição GET para `/agenda` — um F5 depois disso só recarrega a listagem, sem repetir o salvamento.

RESUMO

CRUD: 2 de 4 no bolso. Read e Update, feitos.

- 1 Hoje: repositório em memória (DAO) + tela de Editar Contato
- 2 Próxima aula: Create e Delete — novo contato e excluir contato
- 3 Depois: substituir o DAO em memória por JDBC de verdade